Python Regular Expressions (Regex) - Quick Reference Handout

Single-page student reference for the Python re module

1) What is a regex?

A regular expression (regex / regexp) is a pattern used to match text. Use regex to find, check, extract, and replace text. In Python, regex is handled by the re module.

```
import re
```

Write patterns as raw strings where possible, e.g. r"\d+", to avoid backslash escaping problems.

2) Main Python regex functions

- re.search(pattern, text) - find the **first** match anywhere in the string
  - re.search(r"hello", "well, hello there") → hello
- re.match(pattern, text) - **match only** at the start of the string
  - re.match(r"well", "well, hello there") → well
- re.fullmatch(pattern, text) - match the **whole** string
  - re.fullmatch(r"[A-Z]{5}", "ABCDE") → ABCDE
- re.findall(pattern, text) - return **all matches** as a list
  - re.findall(r"\d+", "Order 17 costs 42 pounds") → ['17', '42']
- re.sub(pattern, replacement, text) - **replace** matching text
  - re.sub(r"\s+", " ", "too many spaces") → 'too many spaces'
- re.split(pattern, text) - **split text** using a regex pattern
  - re.split("w", "hello-world") → ['hello-', 'orld']

3) Useful regex symbols

Basic patterns: hello = exact text; .(DOT) = any single character

```
re.findall(r"c.t", "cat cot cut") → ['cat', 'cot', 'cut']
```

Character classes: [abc] one of a, b, c; [A-Z] uppercase letter; [0-9] digit; [^0-9] not a digit

```
re.findall(r"[A-Z]", "Hello There") → ['H', 'T']
```

Shorthand classes: \d digit, \D non-digit, \w word character, \W non-word character, \s whitespace, \S non-whitespace

```
re.findall(r"\d+", "A12 B7 C300") → ['12', '7', '300']
```

4) Quantifiers, anchors, and boundaries

* = 0 or more, + = 1 or more, ? = 0 or 1, {n} = exactly n, {n,} = at least n, {n,m} = between n and m

```
re.findall(r"ab*", "a ab abb") → ['a', 'ab', 'abb']
re.findall(r"ab+", "a ab abb") → ['ab', 'abb']
re.findall(r"colou?r", "color colour") → ['color', 'colour']
re.findall(r"\d{2}", "12 4 456") → ['12', '45']
```

^ = start of string, $ = end of string, \b = word boundary

```
re.findall(r"\bcat\b", "cat scatter cat") → ['cat', 'cat']
```

Alternation: | means OR. Grouping: (...) groups parts of a pattern and can capture sub-parts.

```
re.findall(r"happy|elated|joy", "Alice was elated. Bob was happy. Charlie felt joy")
→ ['elated', 'happy', 'joy']

m = re.search(r"(\d{4})-(\d{2})-(\d{2})", "2026-06-09")
print(m.group(1), m.group(2), m.group(3)) → 2026 06 09
```

5) Flags

re.IGNORECASE ignores case;

```
re.findall(r"hello", "Hello HELLO", re.IGNORECASE) → ['Hello', 'HELLO']
```

re.DOTALL makes . match newlines; re.MULTILINE makes ^ and $ work line by line.

6) Common practical examples

- Find all numbers: re.findall(r"\d+", "Order 17 costs 42 pounds") → ['17', '42']
- Validate a simple code: re.fullmatch(r"[A-Z]{2}\d{2}", "AB12") → 'AB12'
- Replace extra character: re.sub(r"%+", " ", "too%%many%%symbols") → 'too many symbols'
- Extract a date: re.findall(r"\d{4}-\d{2}-\d{2}", "Date: 2026-06-09") → ['2026-06-09']

7)Match objects

re.search() and re.match() return a match object if successful, otherwise None. Useful methods: m.group(), m.span(), m.start(), m.end().

```
m = re.search(r"cat", "The cat sat")
print(m.group())   → cat
print(m.span())    → (4, 7)
```

8) Common mistakes

- Forgetting raw strings: prefer r"\d+" to "\\d+".
- Using match() when you need search().
- Forgetting that matching is case-sensitive unless you use re.IGNORECASE.

9) Mini summary

Use search() for patterns anywhere, match() at the start, fullmatch() for whole-string validation, findall() to get all matches, sub() to replace, and split() to split text.

Most important symbols to remember: . \d \w \s [] ^ $ \b * + ? {n,m} | ()